pfi

Python for Fixed Income

COURSE 9 · DEPLOYMENT

FastAPI: A Bond Math Service

WEEK 2 · DAY 1

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Today: a bond math service

Today

1. What a service is: request, response, status
2. The request and response models
3. bond_app: four endpoints and a schema
4. The test client: no port, no process
5. A bad request: a 422 naming the field
6. One real server, started and stopped
7. What FastAPI does not check
8. Five bonds: the service, bondmath, Excel
9. Every setting of the app, the route, the server

Three packages

`fastapi`

the web framework

`pydantic`

checks each request

`uvicorn`

the server program

Today the service runs on your machine. Day 7 puts it at a public URL.

What the service returns, and what it never says

Numbers in, numbers out	units in every key: price_per_100, dv01_per_100
Synthetic bonds	the week's bond and five bonds of the synthetic book
It computes	it holds no data and never says a number is high or low

The service's outputs

Allowed names

price per 100 of face

DV01 per 100 of face

the service returned

Never called

fair value, cheap, or rich

a quote or a mark

a recommendation

What a service is

On your laptop

```
bondmath.dv01(
  dates, 5.0, 5.06, 2)
0.043625
```


Through a service

```
POST /dv01
{"settlement": "2026-10-02", "maturity": "2031-10-02", ...}
status 200  {"dv01_per_100": 0.043624879198739 52}
```

method	GET reads; POST sends a body
path	/dv01: which function
body	JSON text: the arguments
status	200 success; 422 refused

How the request models work

The problem	A request body is text from someone else's machine; check it before any bond math runs
The model	A pydantic class names each field with its type and rules; FastAPI parses the body into it
A misfit	A 422 that names the field; a strict field refuses a number sent as text
You control	Which fields exist, which are required, how strict each is
What comes out	A typed object the route can trust, or a refusal that never reaches bondmath

The request model

```
class BondRequest(pydantic.BaseModel):
    settlement: date
    maturity: date
    coupon_pct: float  (strict)
    yield_pct: float  (strict)
    frequency: Literal[1, 2, 4]
```

Three rules

Every field required: no default

Numbers strict: "5.06" refused

Maturity after settlement

```
refused: 1 problem; field ('yield_pct',), missing: Field required
```

Response models hold one number each, unit in the name: price_per_100, dv01_per_100.

How an app answers a request

The problem	A service must turn a method and a path into the right function, every time
The routes	Each @app.post line joins a method and a path to the function under it
A request	FastAPI finds the route, parses the body into the argument's type, runs the function
You control	The path, the argument model, and the return model
What comes out	An app that answers the routes, writes its schema, and serves /docs

bond_app: routes and a schema

```
GET   /health   Health
POST  /price    Price
POST  /accrued  Accrued
POST  /dv01     Dv01
POST  /yield    Yield To Maturity
```

```
required: ['settlement', 'maturity', 'coupon_pct', 'yield_pct', 'frequency']
settlement  {'type': 'string', 'format': 'date'}
maturity  {'type': 'string', 'format': 'date'}
coupon_pct  {'type': 'number'}
yield_pct  {'type': 'number'}
frequency  {'type': 'integer', 'enum': [1, 2, 4]}
```

FastAPI writes the schema from your models; a running service shows it at /docs.

GET /health: {"status": "ok", ...}; as_of None, the service holds no data

A Flask route and a FastAPI route

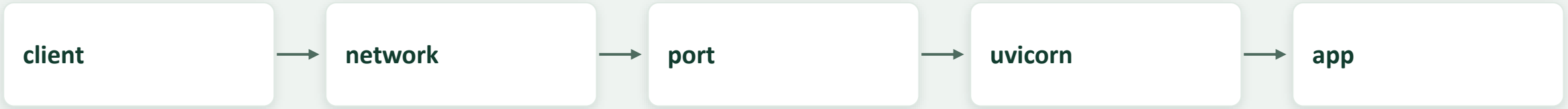
Piece of the job	In Flask	In this course's FastAPI
declare a route	@app.route("/dv01", methods=["POST"])	@app.post("/dv01")
read the body	data = request.get_json()	the argument bond: BondRequest
check the body	by hand: test each key, build the error yourself	automatic: a 422 naming the field
build the answer	return jsonify({"dv01_per_100": value})	return DV01Response(...): the return type shapes the JSON
set a status	return jsonify(...), 400	200 unless the model refuses (422) or status_code= says otherwise
describe the service	not built in	the schema FastAPI writes from the models, and the /docs page
test without a port	app.test_client()	TestClient(app)
run a server	flask run, or a WSGI server such as gunicorn	uvicorn, an ASGI server

The same job. The difference is who does the checking: here the model is the argument.

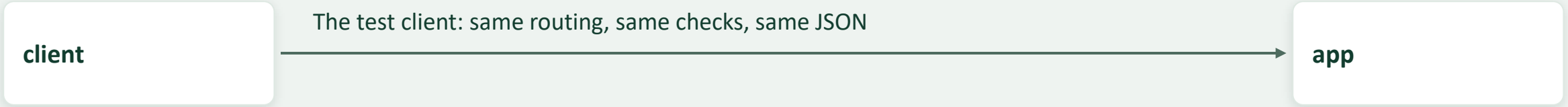
How the test client works

The problem	Testing a service should not need a port, a network, or a second program
The method	It builds the HTTP request and hands it to the app through ASGI, in the same process
What is real	The routing, the model checks, and the JSON; only the network is missing
You control	The method, the path, and the body
What comes out	A response with a status code and a body to test

The test client: no port, no process



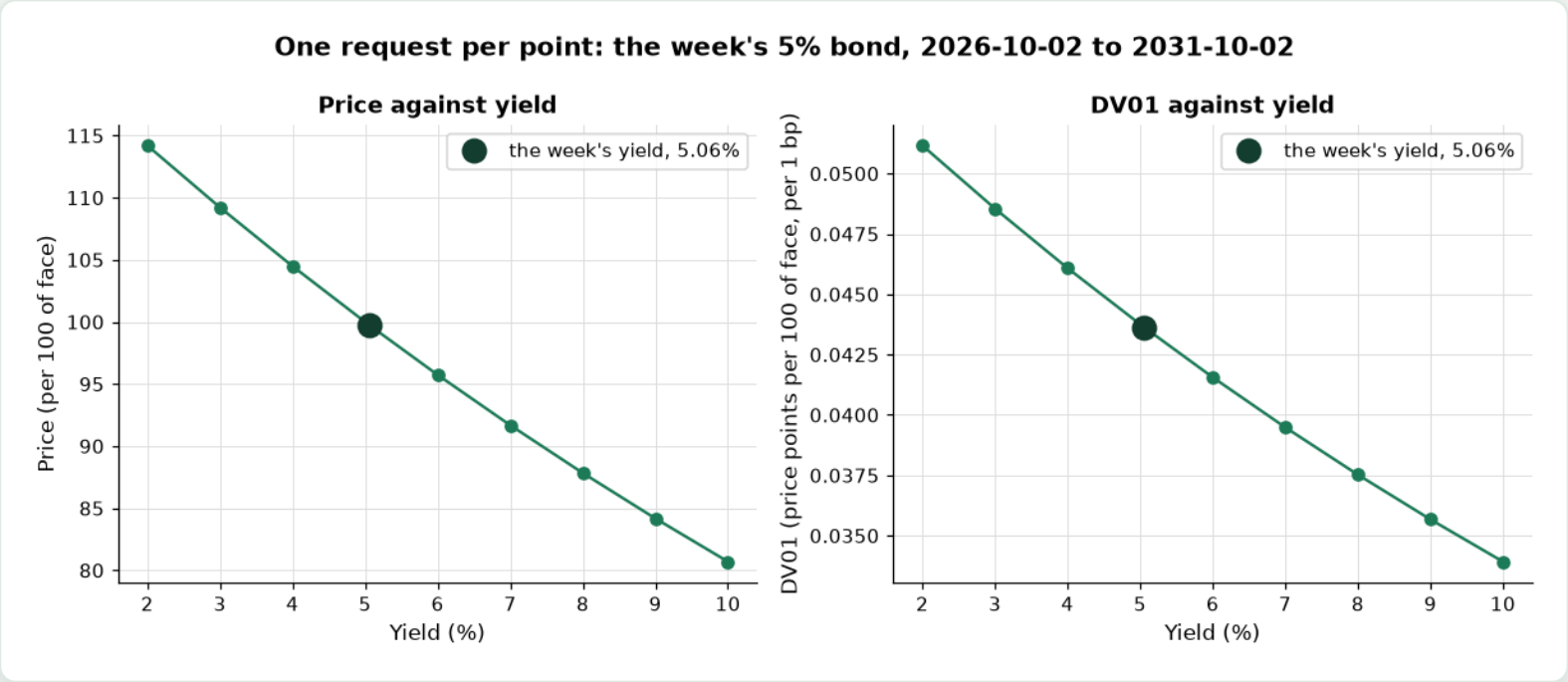
A real server




```
the service: 0.04362487919873952
bondmath:    0.04362487919873952
```

Equal to the last digit. 22 tests pass, none opens a port.

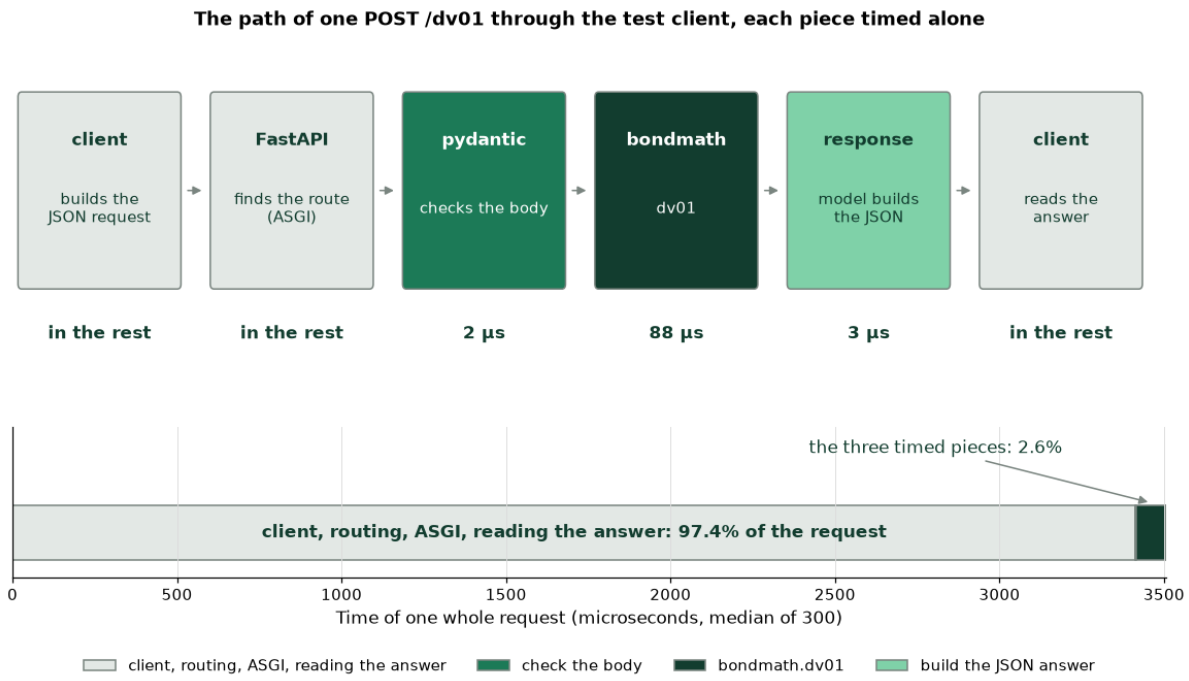
The week's bond, drawn through the service



Yield (%)	Price	DV01
2	114.2070	0.0512
4	104.4913	0.0461
5.06	99.7378	0.0436
6	95.7349	0.0416
8	87.8337	0.0375
10	80.6957	0.0339

One request per point. The large dot is the week's own request, at 5.06%.

The path of one request



Reading it

Green: timed alone

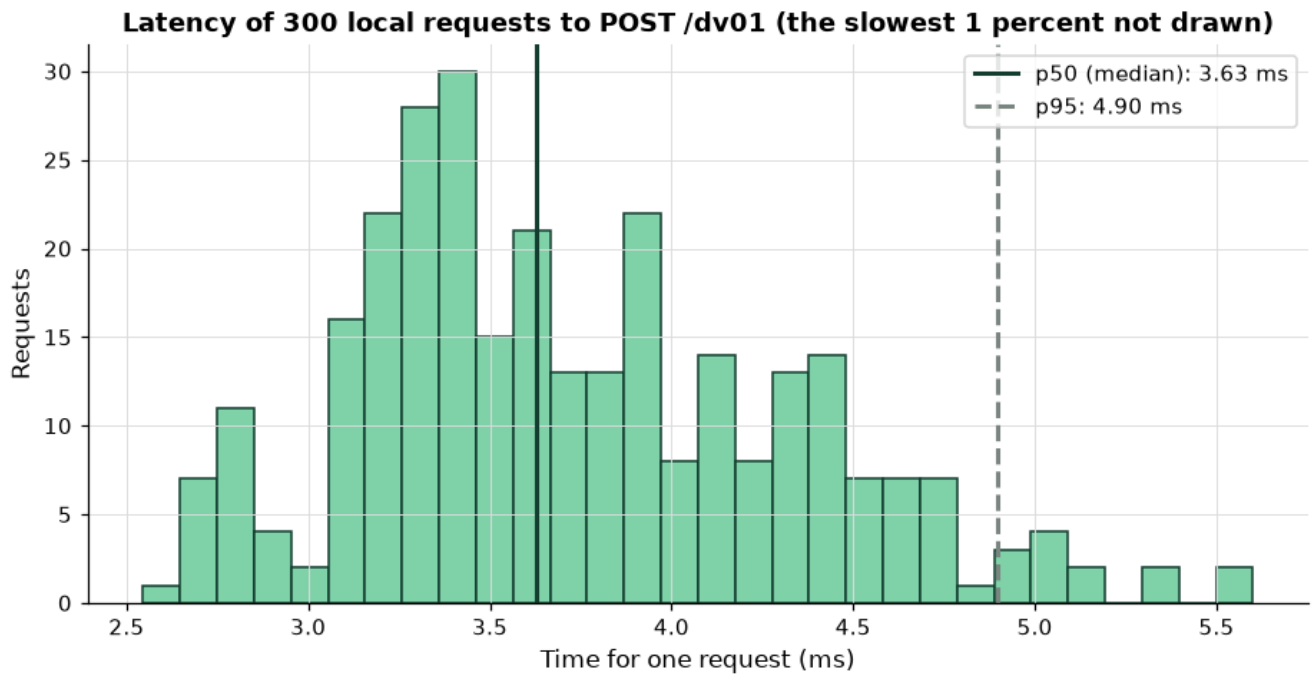
Grey: everything else

The bar: the whole request

Medians of 300 repeats; they move run to run, so they are on the chart only.

The bond math is a sliver of the request; the machinery around it is nearly all of it.

Latency: p50 and p95



Two numbers

p50

half took less

p95

19 in 20 took less

Local requests, no network. A deployed service adds the network and a cold start.

A bad request is a 422 that names the field

```
the yield left out: status 422; field ['body', 'yield_pct']
the coupon as text: status 422; field ['body', 'coupon_pct']
settlement after maturity: status 422; field ['body', 'maturity']
```

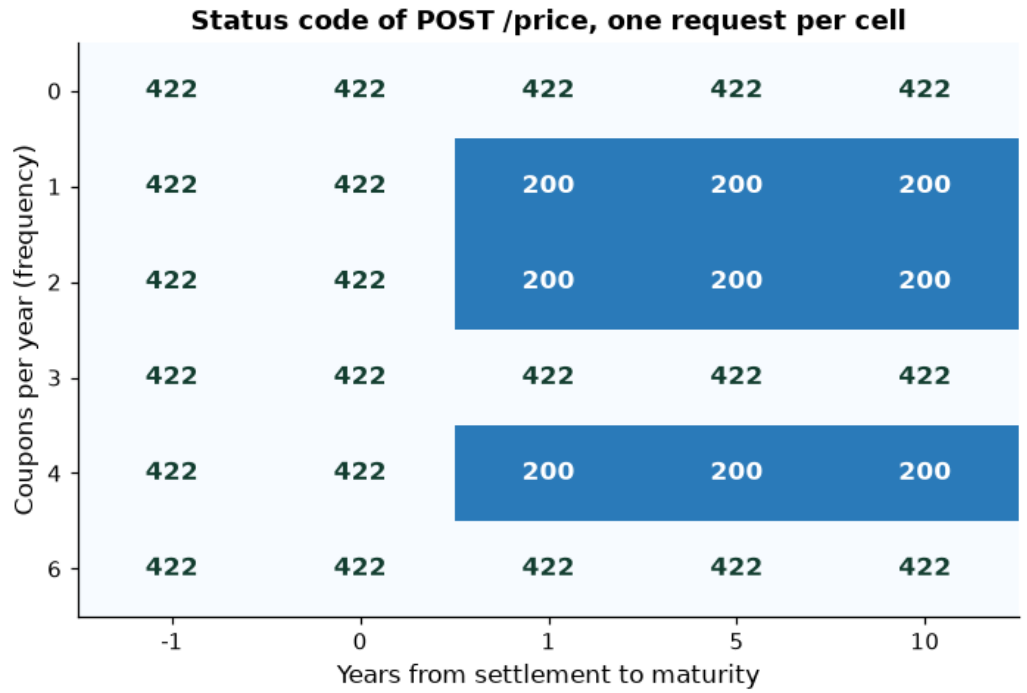
```
Field required

Input should be a valid number

Value error, maturity must be after settlement
```

bondmath never ran. The 422 body echoes the input: one reason a request log never stores bodies.

Which requests the model accepts



Request	Status	Field	Problem
the yield left out	422	yield_pct	missing
the coupon as text	422	coupon_pct	float_type
settlement after maturity	422	maturity	value_error

Nine of thirty accepted: frequency 1, 2, or 4 and a maturity after the settlement date.

The service's contract

Request	Status	Meaning	Body holds
GET /health	200	success	as_of, status, version
POST /dv01, a valid bond	200	success	dv01_per_100
POST /dv01, yield left out	422	the body does not fit the model	detail
POST /dv01, coupon as text	422	the body does not fit the model	detail
GET /dv01, the wrong method	405	the path exists, not for this method	detail
POST /nowhere, an unknown path	404	no such path	detail

```
{'type': 'missing', 'loc': ['body', 'yield_pct'], 'msg': 'Field required'}
```

Check the status first, then read the body; the loc of a 422 says what to fix.

How a server program works

The problem	Bytes arrive on the network; the app needs requests it can answer
The method	uvicorn opens a port, reads each HTTP request off a connection, hands it to the app in ASGI form
The answer	It writes the app's response back to the same connection
You control	The host (127.0.0.1 is this machine only), the port, how much it logs
What comes out	An address any client can send HTTP to; on Vercel the platform runs this

One real server, started and stopped in one cell

port held? refuse → start uvicorn → wait for /health → hand over the URL → stop; port free

```
the server answers at http://127.0.0.1:8765
GET /health: status 200, {'status': 'ok', 'version': '1.0', 'as_of': None}
POST /dv01: status 200, {'dv01_per_100': 0.04362487919873952}
equal to bondmath.dv01 to the last digit: True
a second server: refused, port 8765 is already in use on 127.0.0.1
port 8765 free after the block: True
the server's log: 0 bytes
```

127.0.0.1 is this machine only. The rest of the course uses the test client.

What FastAPI does not check

```
yield_pct: 506.0   (meant 5.06)
status 200; price_per_100 0.988471856785912
```

the shape

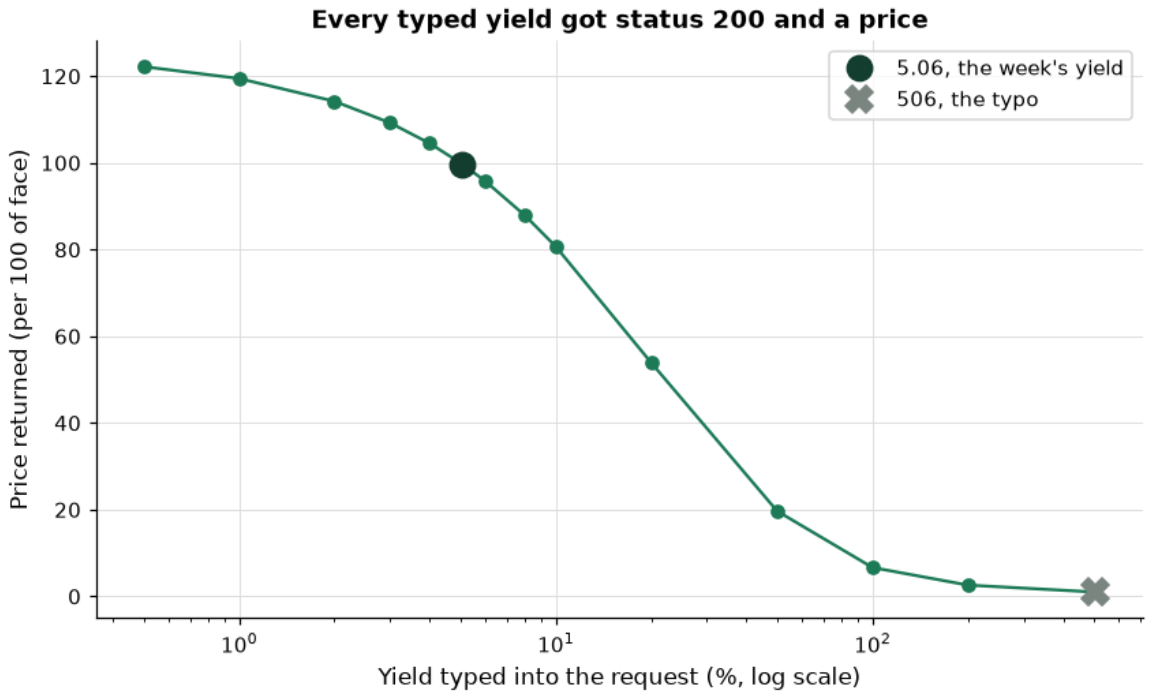
fields, types, dates: the model

the numbers

a yield's sense: your tests

FastAPI 0.143.0 (2026-10-08): automatic OpenTelemetry setup is opt-in. The course never switches it on.

The typo on the curve



Yield typed (%)	Status	Price
5.06	200	99.7378
50	200	19.6637
100	200	6.6474
506	200	0.9885

Every request on the curve got status 200. The tests check the numbers; the framework checks the shape.

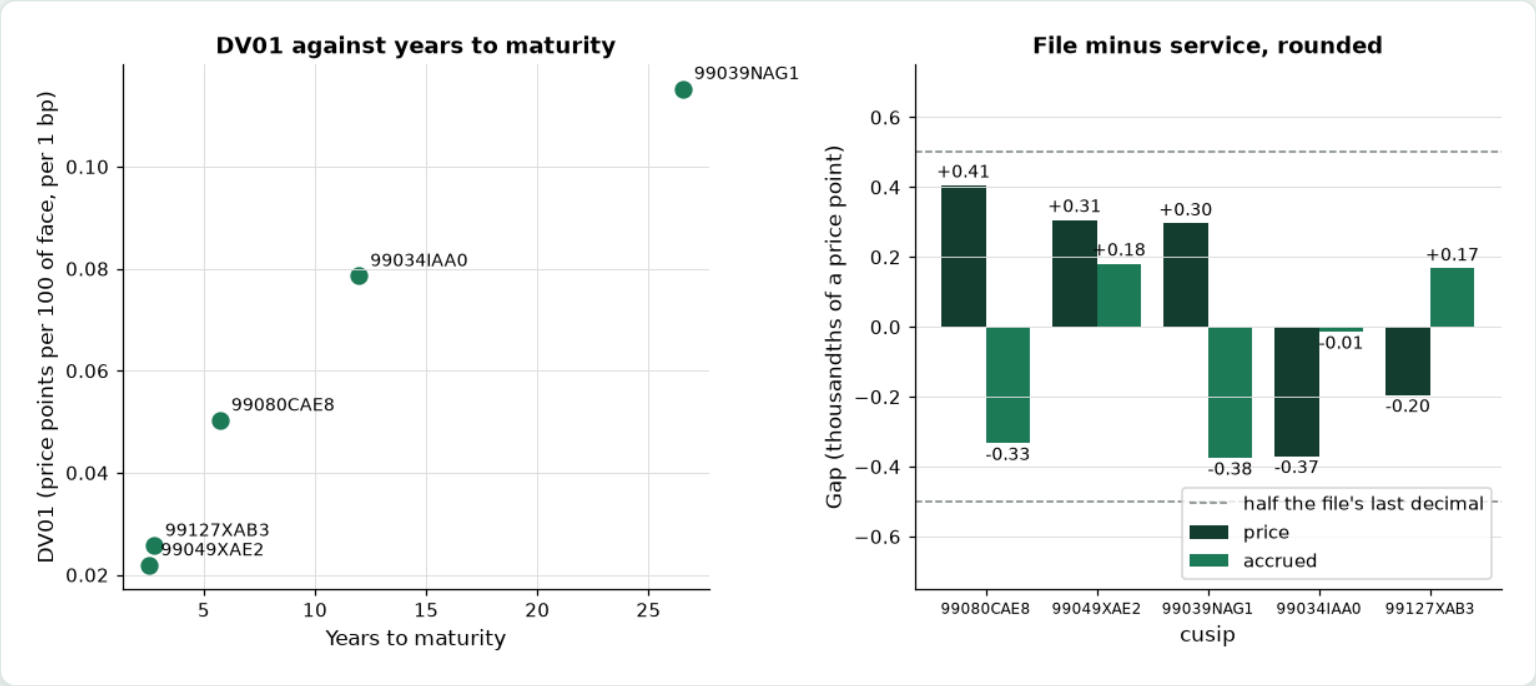
Five bonds, the service, and Excel

cusip	maturity	coupon	yield	price/100	DV01/100	= bondmath
99080CAE8	2032-06-18	7.000	5.606	106.726595	0.050300	True
99049XAE2	2029-04-17	8.125	9.220	97.555694	0.021915	True
99039NAG1	2053-04-09	6.125	6.756	92.260704	0.115277	True
99034IAA0	2038-09-17	4.625	5.960	88.692370	0.078608	True
99127XAB3	2029-06-26	5.250	4.590	101.673195	0.025742	True

```
A2  =PRICE(DATE(2026,9,30),DATE(2032,6,18),0.07,0.05606,100,2,0)
B2  =100*0.07/2*COUPDAYBS(...)/COUPDAYS(...)
C2  =MDURATION(DATE(2026,9,30),DATE(2032,6,18),0.07,0.05606,2,0)*(A2+B2)/10000
```

Excel: prices within 3e-14, DV01s within 3e-17; equal at 6 decimals, all six bonds.

Five bonds, drawn



CUSIP	Maturity	Price	DV01
99080CAE8	2032-06-18	106.7266	0.0503
99049XAE2	2029-04-17	97.5557	0.0219
99039NAG1	2053-04-09	92.2607	0.1153
99034IAA0	2038-09-17	88.6924	0.0786
99127XAB3	2029-06-26	101.6732	0.0257

Every gap from the file is inside half of the file's last decimal.

How the settings work

The problem	A service has many behaviours: what it is called, where its docs live, how its server runs
Where they live	FastAPI(...) for the app, @app.post(...) for a route, uvicorn Config(...) for the server
The defaults	The library's, read from the installed version; the course sets few
You control	A setting, by passing it
What comes out	A service that behaves the way its address, docs page, and logs say

The settings the course names

FastAPI: 39 settings

`title`
PFI bond service, the name on the /docs page

`version`
'1.0', the version /health reports and the schema shows

@app.post: 23 settings

`path`
set on every route: /price, /accrued, /dv01, /yield

uvicorn: 53 settings

`app`
service:app, the two lines of service.py

`host`
127.0.0.1, so nothing outside this machine can connect

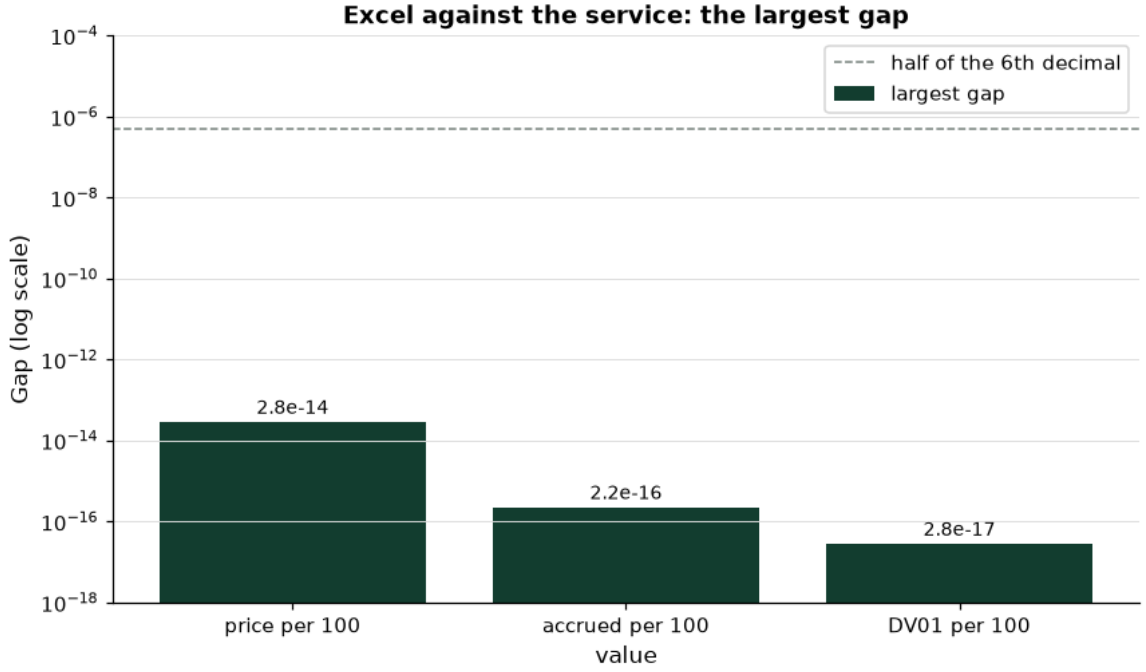
`port`
8765 in section 6.6

`log_level`
warning, so a clean run writes nothing

`access_log`
off (--no-access-log), one less file to clean up

Every other setting is the library's default. On Vercel the platform runs the server.

Excel against the service, drawn



Value	Largest gap
price per 100	2.8e-14
accrued per 100	2.2e-16
DV01 per 100	2.8e-17

At 6 decimals, equal: every bar is far under half of the 6th decimal.

Excel and Git

WEBSERVICE on the running service

```
/health: {"status":"ok","version":"1.0","as_of":null}

/dv01: #VALUE!   WEBSERVICE sends GET only; /dv01 takes POST
```

```
deploykit day 6: the bond service (bond_app, serve) and service.py, 25 tests
```

```
.gitignore:5:uvicorn_*.log   uvicorn_8765.log
```

Six files in one commit; the holdings file and the server's log are ignored.

The AI check

The docstring is the prompt. The deploy code that comes back runs and looks finished.

1

Docstring

Units, conventions, and what the code must never let through: what you asked for.

→

2

Code back

Written for the course in the style of an AI answer. Read it.

→

3

Test first

Plant the case the docstring promises to catch. Run it: it fails.

→

4

Fix

Find the rule it broke, fix it, run the test again.

Today: yield_pct: float = 5.0, a default: a body that forgot the yield is priced at 5% with status 200.

Code that runs is not the same as code that is right.

What you can do now

- A service: method, path, body, status, JSON
- Models: every field required, units in names
- Test every endpoint with the test client
- Read a 422 and the field it names
- Start and stop one real server with serve

Week 2, Day 2 Deploying the service to Vercel